

Tic Tac Toe AI: A Theoretical Exploration of Minimax Implementation

This document outlines the theoretical underpinnings of a Tic Tac Toe game implemented with an AI opponent utilizing the Minimax algorithm, developed using Python and the Pygame library. This project emphasizes the principles of game theory, AI decision-making, and efficient software design.

1. Game Theory and the Tic Tac Toe Paradigm

Tic Tac Toe serves as a classic example in game theory due to its simple rules, finite number of states, and the guarantee of a win, loss, or draw given optimal play. These characteristics make it an ideal environment for exploring AI algorithms. The core concept is *adversarial search*, where the AI (and the opponent) are assumed to be rational actors seeking to maximize their own gain. In the context of Tic Tac Toe, this means maximizing the chances of winning.

2. The Minimax Algorithm: A Foundation for AI Decision Making

At the heart of the AI opponent lies the Minimax algorithm. Minimax is a recursive decision-making algorithm used in game theory for two-player, zero-sum games. Its fundamental principle is to recursively explore all possible future game states from the current state, assuming the opponent will always make the move that is most detrimental to the AI's position.

- **Maximizing Player:** The AI is the maximizing player, striving to choose moves that yield the highest possible score (representing a win or favorable board state).
- **Minimizing Player:** The human opponent is treated as the minimizing player, assumed to always choose moves that minimize the AI's score (leading to a loss or unfavorable state for the AI).

The algorithm recursively explores the game tree to a certain *depth*. The depth represents how many moves ahead the AI is looking. At each level of the tree, the algorithm alternates between maximizing and minimizing. At the terminal nodes (either a game-over state or the maximum depth), a heuristic *evaluation function* is used to assign a numerical score to the board state. These scores are then propagated back up the tree, with the maximizing player choosing the maximum score of its children and the minimizing player choosing the minimum score. The move corresponding to the highest score at the root node is selected as the AI's optimal move.

3. The Evaluation Function: Guiding the Search

The *evaluation function* is a critical component of the Minimax algorithm. It provides a way to estimate the value of a board state when the algorithm cannot search all the way to the end of the game (due to computational limitations). In Tic Tac Toe, a simple evaluation function might:

- Assign a high positive score to a winning state for the AI.
- Assign a high negative score to a winning state for the opponent.
- Assign a score of 0 to a draw state.
- Assign intermediate scores based on factors like the number of rows, columns, or diagonals with two of the AI's marks and zero of the opponent's. These "near-win" states are considered more favorable.

The better the evaluation function, the more accurately the AI can assess the value of different board states and make informed decisions.

4. Software Design and Implementation

The described Tic Tac Toe game utilizes a modular design, separating the game logic, AI implementation, and main game loop into distinct files. This structure promotes code reusability, maintainability, and testability.

- **src/game.py**: Encapsulates the core game logic, including the game board representation, move validation, game over checks, and potentially rendering functions using Pygame.
- **src/minimax.py**: Implements the Minimax algorithm (possibly with alpha-beta pruning) and the evaluation function. It provides a function to determine the AI's optimal move.
- **play.py**: Serves as the main entry point, managing the game loop, handling user input (mouse clicks), calling the AI, and rendering the game state using Pygame.

7. Conclusion

This Tic Tac Toe game demonstrates the application of the Minimax algorithm (and its optimization, alpha-beta pruning) to create an AI opponent. The modular design, coupled with the use of Pygame for rendering and user interaction, provides a robust and engaging implementation of this classic game theory problem. This implementation also serves as a valuable foundation for exploring more complex AI algorithms in other game environments. The strength of the AI is directly related to the depth of the Minimax search and the quality of the evaluation function. Deeper searches and more accurate evaluations lead to more intelligent AI behavior.

8. Minmax Function Implementaion in code

```
def minimax(minimax_board , depth , is_maximizing):

    if check_win(2, minimax_board):

        return float("inf")

    elif check_win(1, minimax_board):

        return float("-inf")

    elif is_board_full(minimax_board):

        return 0

    if is_maximizing:

        best_score = -1000

        for row in range(board_rows):

            for col in range(board_cols):

                if minimax_board[row][col] == 0 :

                    minimax_board[row][col] = 2

                    score = minimax(minimax_board, depth +1 , False)

                    minimax_board[row][col] = 0

                    best_score = max(score ,best_score)

            return best_score

    else :

        best_score = 1000

        for row in range(board_rows):

            for col in range(board_cols):

                if minimax_board[row][col] == 0 :

                    minimax_board[row][col] = 1

                    score = minimax(minimax_board, depth +1 , True)

                    minimax_board[row][col] = 0

                    best_score = min(score ,best_score)

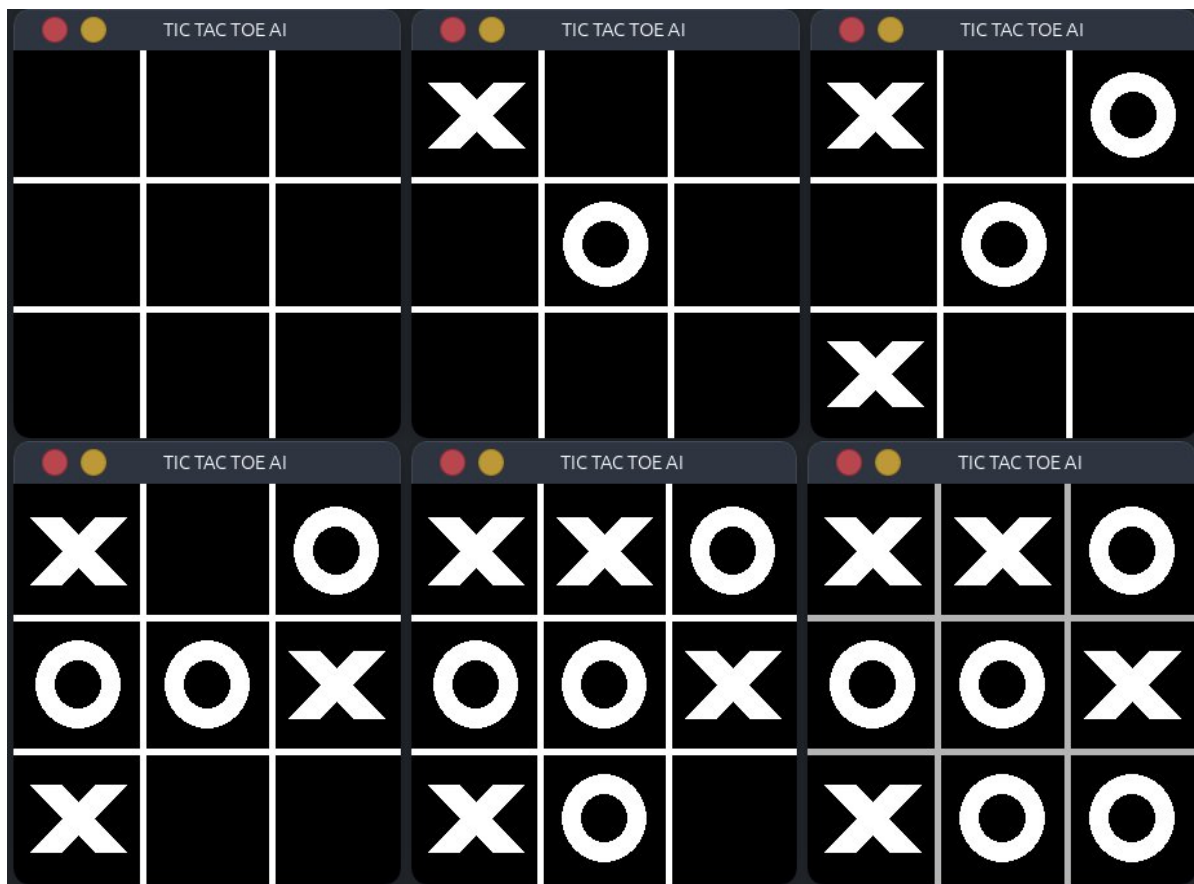
            return best_score
```

How it Works:

1. **Base Cases:** If someone wins or the board is full, return a score (win = infinity, loss = -infinity, draw = 0).
2. **Maximizing Player:** Try each possible move, recursively call minimax to see what the opponent would do, and pick the move that leads to the highest score.
3. **Minimizing Player:** Same as maximizing, but pick the move that leads to the *lowest* score for the opponent.
4. **Backtrack:** After each attempted move, undo it to explore other possibilities.

In essence: It simulates the game to see what would happen if each player made the best possible moves, then chooses the best option for the current player.

9. Game Gui And Game Play Demonstration



9. Code Output In Terminal

```
(academics) pawan@vajra:~/dev/Academics/5TH_SEM/AI/practicals/04/B_tic-tac-toe$ python play.py
pygame 2.6.1 (SDL 2.28.4, Python 3.12.11)
Hello from the pygame community. https://www.pygame.org/contribute.html
It's a Tie!
```